CRONOGRAMA DETALLADO PARA JIRA

QuindioFlix — Plataforma de Streaming

Proyecto Final — Bases de Datos II

Universidad del Quindío | Semestre 2026-1 | Semanas 8–15

---

1. Información General del Proyecto

Nombre del Proyecto	QuindioFlix — Plataforma de Streaming de Contenido Multimedia
Tipo	Scrum
Equipo	Daniel Narváez, Diego García, Cristhian Osorio
Semana actual	Semana 7 — Viernes 20 de marzo de 2026
Inicio del trabajo	Semana 8 — 22 al 29 de marzo de 2026 (arranque desde cero)
Fecha de entrega final	Semana 15 — 18 al 24 de mayo de 2026
Entorno técnico	Java springboot + flyway(IntelliJ IDEA) + Oracle XE 21c local — Solo backend, sin despliegue ni frontend
Temas vistos hasta ahora	Consultas avanzadas, Tablespaces, Datafiles, Fragmentación (RANGE, HASH, LIST) — Hoy (S7) inicia PL/SQL

---

2. Calendario del Semestre

Semana	Período	Sprint	Tema en clase / Avance proyecto
S8	22/03 – 29/03	Sprint 1 (inicio)	creación proyecto + MER + Modelo físico + Tablespaces + Datos de prueba
⛪ **Semana Santa**	**30/03 – 05/04**	**— Receso —**	**Sin actividades académicas**
S9	06/04 – 12/04	Sprint 1 (cierre)	Finalizar CREATE TABLE + datos de prueba completos
S10	13/04 – 19/04	Sprint 2 (inicio)	PIVOT, UNPIVOT — Iniciar consultas avanzadas
S11	20/04 – 26/04	Sprint 2 (cierre)	ROLLUP, CUBE, GROUPING SETS — Completar NT1
S12	27/04 – 03/05	Sprint 3 (inicio)	Vistas materializadas — Iniciar PL/SQL cursores
S13	04/05 – 10/05	Sprint 3 (cierre)	PL/SQL: SPs, funciones — Completar NT2
S14	11/05 – 17/05	Sprint 4	Triggers + Excepciones + Transacciones + Índices

Semana	Período	Sprint	Tema en clase / Avance proyecto
S15	18/05 – 24/05	Sprint 5	Usuarios y roles — Entrega final y sustentación

3. Épicas (Epics)

ID	Nombre	Descripción
EPIC-0	Setup del proyecto	Proyecto Spring Boot + Flyway, repositorio GitHub, configuración de entornos locales de cada integrante
EPIC-1	Modelo de datos y almacenamiento	MER, modelo físico, tablespaces, datafiles, datos de prueba
EPIC-2	Consultas avanzadas (NT1)	Consultas parametrizadas, PIVOT, UNPIVOT, ROLLUP, CUBE, GROUPING SETS, vistas materializadas, fragmentación
EPIC-3	PL/SQL (NT2)	Cursores, procedimientos almacenados, funciones, excepciones, triggers
EPIC-4	Transacciones y concurrencia (NT3)	Especificación de 3 transacciones críticas + escenario de concurrencia con SELECT FOR UPDATE
EPIC-5	Índices y rendimiento (NT4)	Creación de índices justificados + análisis EXPLAIN PLAN antes/después
EPIC-6	Usuarios y roles (NT5)	Roles Oracle, usuarios, privilegios GRANT, PROFILE de recursos

4. Resumen de Sprints

Sprint	Inicio	Duración	Daniel	Diego	Cristhian
Sprint 1	S8+S9: 22/03–12/04*	2 sem*	Daniel 16h	Diego 19h	Cristhian 21h
Sprint 2	S10+S11: 13/04–26/04	2 sem	Daniel 16h	Diego 13h	Cristhian 12h
Sprint 3	S12+S13: 27/04–10/05	2 sem	Daniel 22h	Diego 23h	Cristhian 19h
Sprint 4	S14: 11/05–17/05	1 sem	Daniel 12h	Diego 12h	Cristhian 14h
Sprint 5	S15: 18/05–24/05	1 sem	Daniel 12h	Diego 11h	Cristhian 12h

Distribución perfectamente equilibrada: exactamente 78h por persona en 8 semanas de trabajo (≈9.75h/semana). * Sprint 1 abarca 2 semanas calendario pero solo 1 semana efectiva por Semana Santa (30/03–05/04). Daniel tiene tareas de setup (SET-1, SET-2, SET-5) que compensan tener menos horas en ese sprint.

5. Sprints Detallados

🔴 **BLOQUEANTE: La tarea B NO puede empezar hasta que A esté terminada**	🟡 **PREFERIBLE: Es mejor hacer A antes que B, pero no es obligatorio**	🟢 **INDEPENDIENTE: Se puede hacer en paralelo sin problemas**

SPRINT 1 — Setup del Proyecto + Modelo de datos + Almacenamiento | S8 (22/03–29/03) + S9 (06/04–12/04) — pausa Semana Santa	
🎯 **Objetivo:** Crear el proyecto Spring Boot + Flyway en GitHub, diseñar el MER, implementar el modelo físico en Oracle con tablespaces en disco F y poblar con datos de prueba asimétricos.	⏱ **Duración:** 2 semanas 📦 **Entregable:** Repo GitHub funcional + MER + Scripts SQL integrados en Flyway + Datos de prueba

EPIC-0: Setup del Proyecto Spring Boot + GitHub

ID	Tarea	Responsable	Est.	Prioridad	Descripción
SET-1	**Crear proyecto Spring Boot base + configurar Flyway**	Daniel	3h	🔴 Crítica	*Crear proyecto en IntelliJ (Spring Initializr): dependencias Spring Boot, Flyway, ojdbc11. Configurar application.properties con conexión a Oracle XE local. Crear estructura de carpetas db/migration/. Verificar que el proyecto arranca sin errores.*
SET-2	**Crear repositorio GitHub y subir proyecto base**	Daniel	2h	🔴 Crítica	*Crear repo en GitHub (quindioflix-bd2). Agregar .gitignore para excluir application.properties (cada quien tiene su conexión local). Subir el proyecto base. Agregar application.properties.example con la plantilla de conexión vacía.*
SET-3	**Clonar repo y configurar entorno local — Diego**	Diego	1h	🔴 Alta	*Hacer clone del repo. Copiar application.properties.example → application.properties y poner sus credenciales de Oracle XE local. Arrancar el proyecto y verificar que Flyway conecta sin errores. Confirmar a Daniel que todo OK.*
SET-4	**Clonar repo y configurar entorno local — Cristhian**	Cristhian	1h	🔴 Alta	*Hacer clone del repo. Copiar application.properties.example → application.properties y poner sus credenciales de Oracle XE local. Arrancar el proyecto y verificar que Flyway conecta sin errores. Confirmar a Daniel que todo OK.*
SET-5	**Definir convención de nombres y estructura de scripts**	Daniel	1h	🟡 Media	*Acordar y documentar en el README: prefijos de archivos (V1__, V2__...), orden de ejecución, cómo cada quien agrega sus scripts. Ejemplo: V3__datos_prueba.sql, V4__nucleo1_consultas.sql, etc.*

EPIC-1: Modelo de Datos y Almacenamiento

ID	Tarea	Responsable	Est.	Prioridad	Descripción
MOD-1	**Diseñar MER completo (draw.io / Oracle DM)**	Daniel	10h	🔴 Crítica	*Identificar todas las entidades del negocio QuindioFlix: USUARIOS, PERFILES, CONTENIDO, REPRODUCCIONES, PAGOS, EMPLEADOS, etc. Modelar cardinalidades, relaciones reflexivas y tablas intermedias N:M. Exportar en PNG/PDF y subir al repo en docs/.*
MOD-2	**Definir tablespaces y datafiles — script**	Cristhian	3h	🔴 Alta	*Crear script V1__tablespaces.sql con ts_quindioflix_datos, ts_quindioflix_indices, ts_quindioflix_reprod_2024,*

ID	Tarea	Responsable	Est.	Prioridad	Descripción
	con ruta configurable				*ts_quindioflix_reprod_2025. Usar DEFINE ruta_base al inicio del script — cada integrante asigna su propia ruta local antes de ejecutar. El script NO debe tener rutas hardcodeadas. Documentar en el README qué es la variable y cómo configurarla según el entorno de cada uno.*
MOD-3	**Script CREATE TABLE con restricciones y comentarios**	Diego	8h	🔴 **Crítica**	*Crear V2__create_tables.sql con todas las tablas: PK, FK, CHECK, NOT NULL, DEFAULT. COMMENT ON TABLE y COMMENT ON COLUMN para cada elemento. Asignar tablespace correspondiente. Hacer commit y push para que los demás hagan pull.*
MOD-4	**Fragmentar tabla REPRODUCCIONES por rango de fechas**	Cristhian	4h	🔴 **Alta**	*Incluir en V2__create_tables.sql la creación de REPRODUCCIONES con PARTITION BY RANGE (fecha_hora_inicio): p_2024 → ts_quindioflix_reprod_2024, p_2025 → ts_quindioflix_reprod_2025. Justificar en comentarios del script.*
MOD-5	**Script de datos de prueba — Tablas maestras**	Diego	6h	🔴 **Alta**	*Crear V3__datos_maestros.sql con: 3 PLANES, 5 CATEGORIAS, 8 GENEROS, 5 DEPARTAMENTOS, 10 EMPLEADOS, 40 CONTENIDOS distribuidos en categorías. Commit y push. Flyway lo integra automáticamente al hacer pull.*
MOD-6	**Script de datos de prueba — Tablas transaccionales**	Cristhian	8h	🔴 **Alta**	*Crear V4__datos_transaccionales.sql con: 30 USUARIOS (3 ciudades, 3 planes), 50 PERFILES, 15 TEMPORADAS, 50 EPISODIOS, 200 REPRODUCCIONES, 60 CALIFICACIONES, 80 PAGOS, 40 FAVORITOS. Datos ASIMÉTRICOS. Commit y push.*
MOD-7	**Verificar integridad — arrancar proyecto y revisar**	Diego	4h	🟡 **Media**	*Hacer pull del repo completo. Arrancar el proyecto Spring Boot — Flyway ejecuta V1 a V4 en orden automáticamente. Verificar en IntelliJ Database o SQL Developer que no hay errores de FK, CHECK ni fragmentación.*
MOD-8	**Reglas de negocio y restricciones del dominio (documento)**	Cristhian	5h	🟡 **Media**	*Redactar mínimo 10 reglas de negocio explícitas e implícitas del contexto QuindioFlix. Documentar actores del sistema, procesos principales y restricciones de atributos clave. Subir a docs/ en el repo.*

Mapa de Dependencias — Sprint 1

SPRINT 1 — Dependencias entre tareas		
Tarea	**Tipo**	**Dependencia**
SET-1	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Es lo primero de todo el proyecto. No depende de nada.
SET-1	🟣 **BLOQUEA**	🟣 Bloquea a SET-2, SET-3, SET-4 — Sin el proyecto base nadie puede clonar ni configurar su entorno.
SET-2	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE SET-1 — El repo se crea una vez que el proyecto base está funcionando.
SET-2	🟣 **BLOQUEA**	🟣 Bloquea a SET-3 y SET-4 — Diego y Cristhian necesitan el repo publicado para hacer clone.
SET-3	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE SET-2 — Diego necesita el repo en GitHub para poder clonarlo.
SET-4	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE SET-2 — Cristhian necesita el repo en GitHub para poder clonarlo.

SET-5	🟡 **PREFERIBLE**	🟡 PREFERIBLE después de SET-2 — La convención de nombres se define una vez que el repo existe y todos tienen acceso.
MOD-1	🟡 **PREFERIBLE**	🟡 PREFERIBLE después de SET-1 — El MER puede arrancarse en paralelo al setup; solo necesita estar listo antes de MOD-3.
MOD-1	🟣 **BLOQUEA A**	🟣 Bloquea a MOD-2, MOD-3, MOD-4 — Sin el MER no se pueden definir tablespaces, crear tablas ni fragmentar.
MOD-2	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE MOD-1 — Los tablespaces se diseñan a partir de las tablas del MER.
MOD-2	🟣 **BLOQUEA A**	🟣 Bloquea a MOD-3 — El CREATE TABLE asigna cada tabla a su tablespace; MOD-2 debe estar commiteado.
MOD-3	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE MOD-1 y MOD-2 — Necesita el MER y el script de tablespaces mergeado en el repo.
MOD-3	🟣 **BLOQUEA A**	🟣 Bloquea a MOD-4, MOD-5, MOD-6, MOD-7 — Sin las tablas en el repo nadie puede avanzar con datos ni fragmentación.
MOD-4	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE MOD-2 y MOD-3 — La fragmentación va dentro del mismo script de tablas; MOD-2 y MOD-3 deben estar listos.
MOD-5	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE MOD-3 — Las tablas maestras deben existir en Oracle antes de insertar.
MOD-5	🟣 **BLOQUEA A**	🟣 Bloquea a MOD-6 — Los datos transaccionales tienen FK hacia las tablas maestras ya pobladas.
MOD-6	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE MOD-3 y MOD-5 — USUARIOS, REPRODUCCIONES y PAGOS referencian tablas maestras con datos.
MOD-6	🟣 **BLOQUEA A**	🟣 Bloquea a MOD-7 — La verificación solo es significativa con todos los datos cargados.
MOD-7	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE MOD-5 y MOD-6 — Solo se puede verificar con tablas y datos completos en Oracle.
MOD-8	🟡 **PREFERIBLE**	🟡 PREFERIBLE después de MOD-1 — El documento de reglas es más sólido con el MER ya trazado.

Total Sprint 1: ~56h (Daniel:16h, Diego:19h, Cristhian:21h) — incluye setup + modelo + datos. Pausa Semana Santa 30/03–05/04

SPRINT 2 — Consultas Avanzadas (Núcleo 1) | S10 (13/04–19/04) + S11 (20/04–26/04)	
🎯 **Objetivo:** Implementar todas las consultas avanzadas del NT1: parametrizadas, PIVOT, UNPIVOT, ROLLUP, CUBE, GROUPING SETS y vistas materializadas.	⏱ **Duración:** 2 semanas 📦 **Entregable:** Script NT1 completo (.sql)

EPIC-2: Consultas Avanzadas (NT1)

ID	Tarea	Responsable	Est.	Prioridad	Descripción
NT1-1	**Consultas parametrizadas (mínimo 3)**	Daniel	5h	🔴 **Alta**	*a) Top 10 contenido más reproducido por ciudad (&ciudad). b) Ingresos por plan según mes y año (&mes, &año). c) Calificación promedio por categoría para un género dado (&genero). Usar variables de sustitución (&, &&, DEFINE).*
NT1-2	**PIVOT — Reporte usuarios por ciudad y plan**	Diego	4h	🔴 **Alta**	*Filas = ciudades, columnas = planes (Básico, Estándar, Premium), valores = cantidad de usuarios activos. Resultado legible con etiquetas claras.*

ID	Tarea	Responsable	Est.	Prioridad	Descripción
NT1-3	**PIVOT — Reporte reproducciones por categoría y dispositivo**	Cristhian	4h	🔴 **Alta**	*Filas = categorías de contenido, columnas = dispositivos (celular, tablet, TV, computador), valores = total de reproducciones.*
NT1-4	**UNPIVOT — Convertir reporte de usuarios pivoteado a filas**	Daniel	3h	🟡 **Media**	*Tomar el PIVOT de usuarios por plan y convertirlo de nuevo a filas. Mostrar ciudad, plan y cantidad como columnas normalizadas.*
NT1-5	**UNPIVOT — Convertir tabla de resumen mensual a filas**	Diego	3h	🔴 **Alta**	*Tomar tabla con columnas Enero, Febrero, Marzo, etc. y convertirla a filas: ciudad | mes | valor. Útil para análisis de tendencias.*
NT1-6	**ROLLUP — Ingresos por ciudad y plan con subtotales**	Cristhian	4h	🔴 **Alta**	*GROUP BY ROLLUP(ciudad, plan): subtotales por ciudad y gran total general. Usar GROUPING() para reemplazar NULLs con etiquetas 'SUBTOTAL' y 'TOTAL GENERAL'.*
NT1-7	**CUBE — Reproducciones por categoría y dispositivo**	Daniel	4h	🔴 **Alta**	*GROUP BY CUBE(categoria, dispositivo): todas las combinaciones posibles incluyendo totales cruzados. Usar GROUPING SETS para consolidar resultados.*
NT1-8	**GROUPING SETS — Totales por categoría y por ciudad**	Diego	3h	🟡 **Media**	*GROUPING SETS((categoria), (ciudad)): solo los totales por cada dimensión sin el detalle cruzado. Mostrar diferencia con CUBE.*
NT1-9	**Vista materializada — Contenido más popular**	Cristhian	4h	🔴 **Alta**	*Precalcular total de reproducciones y calificación promedio por contenido. Crear con REFRESH COMPLETE ON DEMAND. Usar como base del reporte 'Contenido Más Popular'. Justificar beneficio de rendimiento.*
NT1-10	**Vista materializada — Ingresos mensuales por ciudad y plan**	Daniel	4h	🔴 **Alta**	*Precalcular ingresos mensuales agrupados por ciudad y plan de suscripción. Usar como base del reporte financiero mensual. Demostrar mejora de rendimiento con consulta directa vs vista.*
NT1-11	**Revisar y comentar todo el script NT1**	Diego	3h	🟡 **Media**	*Agregar comentarios explicativos a cada consulta. Verificar que todas se ejecuten sin error en Oracle XE. Organizar el script con secciones claramente delimitadas.*

Mapa de Dependencias — Sprint 2

SPRINT 2 — Dependencias entre tareas		
Tarea	**Tipo**	**Dependencia**
NT1-1	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — No depende de ninguna otra tarea dentro del sprint.
NT1-2	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Se puede hacer en paralelo con NT1-1, NT1-3 y NT1-5.
NT1-2	🟣 **BLOQUEA A**	🟣 Bloquea a NT1-4 — El UNPIVOT de usuarios toma el resultado del PIVOT NT1-2; debe estar listo primero.
NT1-3	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Puede hacerse en paralelo con cualquier otra tarea del sprint.
NT1-4	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE NT1-2 — El UNPIVOT transforma el resultado del PIVOT de usuarios; NT1-2 debe estar terminado.

NT1-5	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Trabaja sobre una tabla de resumen mensual separada; no depende de ninguna otra tarea del sprint.
NT1-6	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Puede hacerse en paralelo con NT1-1, NT1-2, NT1-3 y NT1-5.
NT1-6	🟣 **BLOQUEA A**	🟣 Bloquea preferiblemente a NT1-8 — GROUPING SETS se entiende mejor después de ver ROLLUP.
NT1-7	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Puede hacerse en paralelo con cualquier otra tarea del sprint.
NT1-7	🟣 **BLOQUEA A**	🟣 Bloquea preferiblemente a NT1-8 — GROUPING SETS se entiende mejor después de ver CUBE.
NT1-8	🟡 **PREFERIBLE**	🟡 PREFERIBLE después de NT1-6 y NT1-7 — GROUPING SETS es extensión conceptual de ROLLUP y CUBE; verlos antes facilita la comprensión.
NT1-9	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Las vistas materializadas no dependen de ninguna consulta del mismo sprint.
NT1-10	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Se puede hacer en paralelo con NT1-9 y cualquier consulta del sprint.
NT1-11	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE NT1-1 a NT1-10 — Solo puede revisarse y comentarse cuando todas las consultas y vistas estén terminadas.

Total Sprint 2: ~41h (≈14h por persona)

SPRINT 3 — PL/SQL — Núcleo 2 | S12 (27/04–03/05) + S13 (04/05–10/05)	
🎯 **Objetivo:** Implementar todos los componentes PL/SQL del NT2: cursores, procedimientos almacenados, funciones, manejo de excepciones y triggers.	⏱ **Duración:** 2 semanas 📦 **Entregable:** Script NT2 completo (.sql)

EPIC-3: PL/SQL (NT2)

ID	Tarea	Responsable	Est.	Prioridad	Descripción
NT2-1	**Cursor — Usuarios con suscripción vencida**	Daniel	5h	🔴 **Alta**	*Cursor explícito que recorra usuarios con más de 30 días sin pago. Generar reporte con: nombre, email, plan, días de mora y monto adeudado. Usar %ROWTYPE y FETCH.*
NT2-2	**Cursor — Popularidad de contenido**	Diego	5h	🔴 **Alta**	*Cursor que recorra CONTENIDO y para cada ítem calcule reproducciones completas (porcentaje >= 90%). Actualizar campo de popularidad en la tabla. Usar cursor con FOR UPDATE.*
NT2-3	**SP_REGISTRAR_USUARIO**	Cristhian	7h	🔴 **Crítica**	*SP que recibe datos del usuario + plan elegido. Valida que email no exista, crea cuenta, crea perfil predeterminado y registra primer pago. Usar transacción con COMMIT/ROLLBACK.*
NT2-4	**SP_CAMBIAR_PLAN**	Daniel	6h	🔴 **Crítica**	*SP que recibe id_usuario y nuevo plan. Valida que el usuario no tenga más perfiles de los que permite el nuevo plan. Actualiza plan y registra cambio. Manejo de excepción personalizada.*
NT2-5	**SP_REPORTE_CONSUMO**	Diego	6h	🔴 **Alta**	*SP que recibe id_usuario y rango de fechas. Genera reporte detallado de reproducciones de cada perfil agrupadas por categoría con totales de tiempo consumido.*

ID	Tarea	Responsable	Est.	Prioridad	Descripción
NT2-6	**FN_CALCULAR_MONTO**	Cristhian	5h	🔴 **Alta**	*Función que recibe id_usuario y retorna monto del próximo mes. Considera plan actual y descuentos por antigüedad: más de 12 meses → 10%, más de 24 meses → 15%.*
NT2-7	**FN_CONTENIDO_RECOMENDADO**	Diego	5h	🟡 **Media**	*Función que recibe id_perfil y retorna título del contenido más afín según géneros más reproducidos por ese perfil. Usar subquery o cursor interno.*
NT2-8	**Excepciones en SP_REGISTRAR_USUARIO**	Diego	3h	🔴 **Alta**	*Excepción personalizada EMAIL_DUPLICADO cuando el email ya existe. Excepción NO_DATA_FOUND cuando el plan no es válido. Mostrar mensajes descriptivos con RAISE_APPLICATION_ERROR.*
NT2-9	**Excepciones en SP_CAMBIAR_PLAN**	Cristhian	3h	🔴 **Alta**	*Excepción personalizada PERFILES_EXCEDIDOS con código de error (-20001) cuando el usuario tiene más perfiles de los que permite el nuevo plan. Documentar código de error.*
NT2-10	**Trigger TRG_VALID_CUENTA en REPRODUCCIONES**	Daniel	4h	🔴 **Alta**	*Trigger BEFORE INSERT a nivel de fila. Verifica que el usuario tenga estado_cuenta = 'ACTIVO'. Si no, rechaza la inserción con RAISE_APPLICATION_ERROR.*
NT2-11	**Trigger TRG_MAX_PERFILES en PERFILES**	Diego	4h	🔴 **Alta**	*Trigger BEFORE INSERT a nivel de fila. Verifica que el usuario no supere el máximo de perfiles según su plan (Básico: 2, Estándar: 3, Premium: 5). Rechazar si excede.*
NT2-12	**Trigger TRG_VALID_CALIFICACION en CALIFICACIONES**	Cristhian	4h	🔴 **Alta**	*Trigger BEFORE INSERT a nivel de fila. Verifica que el perfil haya reproducido al menos el 50% del contenido antes de calificar. Rechazar con mensaje claro si no cumple.*
NT2-13	**Trigger TRG_ACTIVAR_CUENTA en PAGOS**	Daniel	4h	🔴 **Alta**	*Trigger AFTER INSERT a nivel de sentencia. Después de insertar un pago exitoso, actualiza estado_cuenta del usuario a 'ACTIVO' y registra fecha_ultimo_pago. Usar :NEW para obtener datos del pago.*
NT2-14	**Revisar y comentar todo el script NT2**	Daniel	3h	🟡 **Media**	*Agregar encabezados descriptivos a cada bloque PL/SQL. Verificar que todos los objetos se compilen sin errores. Probar cada SP, función y trigger con casos de prueba específicos.*

Mapa de Dependencias — Sprint 3

SPRINT 3 — Dependencias entre tareas		
Tarea	**Tipo**	**Dependencia**
NT2-1	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Puede hacerse en paralelo con cualquier otra tarea del sprint.
NT2-2	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Puede hacerse en paralelo con cualquier otra tarea del sprint.
NT2-3	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — No depende de ninguna otra tarea dentro del sprint.
NT2-3	🟣 **BLOQUEA A**	🟣 Bloquea a NT2-8 — Las excepciones de SP_REGISTRAR_USUARIO se añaden dentro del mismo SP; NT2-3 debe existir primero.
NT2-4	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Puede hacerse en paralelo con NT2-1, NT2-2 y NT2-3.

NT2-4	🟣 **BLOQUEA A**	🟣 Bloquea a NT2-9 — Las excepciones de SP_CAMBIAR_PLAN se añaden dentro del mismo SP; NT2-4 debe existir primero.
NT2-5	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Puede hacerse en paralelo con cualquier otra tarea del sprint.
NT2-6	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — No depende de ninguna otra tarea dentro del sprint.
NT2-6	🟣 **BLOQUEA A**	🟣 Bloquea preferiblemente a NT2-7 — FN_CONTENIDO_RECOMENDADO usa lógica similar a FN_CALCULAR_MONTO; conviene tenerla de referencia.
NT2-7	🟡 **PREFERIBLE**	🟡 PREFERIBLE después de NT2-6 — Comparte lógica de agrupación por géneros; NT2-6 sirve de referencia para la implementación.
NT2-8	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE NT2-3 — Las excepciones se agregan al interior del SP; NT2-3 debe estar compilado.
NT2-9	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE NT2-4 — Las excepciones se agregan al interior del SP; NT2-4 debe estar compilado.
NT2-10	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — El trigger en REPRODUCCIONES no depende de ningún otro objeto de este sprint.
NT2-11	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — El trigger en PERFILES no depende de ningún otro objeto de este sprint.
NT2-12	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — El trigger en CALIFICACIONES no depende de ningún otro objeto de este sprint.
NT2-13	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — El trigger en PAGOS no depende de ningún otro objeto de este sprint.
NT2-14	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE NT2-1 a NT2-13 — La revisión y prueba solo puede hacerse cuando todos los objetos PL/SQL estén compilados sin errores.

Total Sprint 3: ~64h (≈21h por persona)

SPRINT 4 — Transacciones + Índices (NT3 + NT4) | S14 (11/05–17/05)	
🎯 **Objetivo:** Diseñar e implementar las 3 transacciones críticas con SAVEPOINT, documentar el escenario de concurrencia y crear/analizar los índices de rendimiento.	⏱ **Duración:** 1 semana 📦 **Entregable:** Script NT3 + Script NT4 + Capturas EXPLAIN PLAN

EPIC-4: Transacciones y Concurrencia (NT3)

ID	Tarea	Responsable	Est.	Prioridad	Descripción
NT3-1	**Transacción de registro completo**	Daniel	5h	🔴 **Alta**	*Crear usuario + perfil + primer pago en una sola transacción. Si falla cualquier paso → ROLLBACK total. Documentar estados: activa, parcialmente confirmada, confirmada, fallida, abortada.*
NT3-2	**Transacción de renovación mensual**	Diego	6h	🔴 **Alta**	*Para cada usuario activo: verificar vencimiento → calcular monto (FN_CALCULAR_MONTO) → registrar pago → actualizar estado. Usar SAVEPOINT por usuario para que si uno falla no pierda los anteriores.*
NT3-3	**Transacción de eliminación de cuenta**	Cristhian	5h	🔴 **Alta**	*Eliminar en orden: CALIFICACIONES → FAVORITOS → REPRODUCCIONES → PERFILES → PAGOS → USUARIO. Todo o nada: si falla cualquier DELETE → ROLLBACK*

ID	Tarea	Responsable	Est.	Prioridad	Descripción
					completo. Documentar el orden por integridad referencial.
NT3-4	**Escenario de concurrencia — SELECT FOR UPDATE**	Daniel	5h	🔴 **Alta**	*Abrir dos sesiones Oracle simultáneas. Las dos intentan cambiar el plan del mismo usuario. Demostrar bloqueo con SELECT FOR UPDATE. Documentar con capturas: sesión 1 espera a sesión 2, luego COMMIT libera el bloqueo.*
NT3-5	**Documentar transacciones en el informe**	Cristhian	3h	🟡 **Media**	*Incluir diagrama de estados para cada transacción. Describir puntos exactos de COMMIT, ROLLBACK y SAVEPOINT. Explicar por qué cada transacción es crítica para el negocio QuindioFlix.*

EPIC-5: Índices y Rendimiento (NT4)

ID	Tarea	Responsable	Est.	Prioridad	Descripción
NT4-1	**Índice en REPRODUCCIONES(id_perfil, fecha_hora_inicio)**	Diego	3h	🔴 **Alta**	*Crear índice compuesto. Justificar: las consultas de historial de un perfil filtran por id_perfil y ordenan por fecha. Sin índice hace full scan de 200+ registros. Con índice usa index range scan.*
NT4-2	**Índice en USUARIOS(email)**	Cristhian	2h	🔴 **Alta**	*Crear índice único. Justificar: login y validación de duplicados en SP_REGISTRAR_USUARIO buscan por email. Es la consulta más frecuente del sistema. Sin índice → full scan de 30 usuarios.*
NT4-3	**Índice en CONTENIDO(id_categoria, anio_lanzamiento)**	Daniel	2h	🔴 **Alta**	*Crear índice compuesto. Justificar: las búsquedas de catálogo filtran primero por categoría y luego por año. Mejora consultas parametrizadas NT1.*
NT4-4	**Índice adicional a elección — Justificado**	Diego	3h	🟡 **Media**	*Proponer índice adicional basado en las consultas más frecuentes identificadas. Sugerencia: PAGOS(id_usuario, fecha_pago, estado_pago) para el cursor de suscripciones vencidas y SP_REPORTE_CONSUMO.*
NT4-5	**Análisis EXPLAIN PLAN antes/después + capturas**	Cristhian	4h	🔴 **Alta**	*Seleccionar una consulta pesada (ej: reporte de reproducciones con JOIN a USUARIOS y CONTENIDO). Ejecutar EXPLAIN PLAN antes de crear índice → captura. Crear índice → EXPLAIN PLAN de nuevo → captura. Analizar diferencia en cost y cardinalidad.*

Mapa de Dependencias — Sprint 4

SPRINT 4 — Dependencias entre tareas (NT3 + NT4)		
Tarea	**Tipo**	**Dependencia**
NT3-1	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Puede hacerse en paralelo con NT3-2, NT3-3 y NT3-4 dentro del sprint.
NT3-2	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Puede hacerse en paralelo con las demás transacciones del sprint.
NT3-3	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Puede hacerse en paralelo con las demás transacciones del sprint.

NT3-4	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — El escenario de concurrencia no depende de NT3-1, NT3-2 ni NT3-3.
NT3-5	🟡 **PREFERIBLE**	🟡 PREFERIBLE después de NT3-1, NT3-2, NT3-3 y NT3-4 — El documento se enriquece con las 3 transacciones y el escenario ya implementados.
NT4-1	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Puede hacerse en paralelo con NT4-2 y NT4-3.
NT4-1	🟣 **BLOQUEA A**	🟣 Bloquea a NT4-5 — El EXPLAIN PLAN compara antes/después; NT4-1 debe estar creado para la segunda medición.
NT4-2	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Puede hacerse en paralelo con NT4-1 y NT4-3.
NT4-2	🟣 **BLOQUEA A**	🟣 Bloquea a NT4-5 — El EXPLAIN PLAN compara antes/después; NT4-2 debe estar creado para la segunda medición.
NT4-3	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — Puede hacerse en paralelo con NT4-1 y NT4-2.
NT4-3	🟣 **BLOQUEA A**	🟣 Bloquea a NT4-5 — El EXPLAIN PLAN compara antes/después; NT4-3 debe estar creado para la segunda medición.
NT4-4	🟡 **PREFERIBLE**	🟡 PREFERIBLE después de NT4-1, NT4-2 y NT4-3 — El índice adicional se elige basado en los patrones de consulta ya identificados al crear los anteriores.
NT4-5	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE NT4-1, NT4-2 y NT4-3 — El análisis EXPLAIN PLAN requiere los índices ya creados para mostrar la mejora antes/después.

Total Sprint 4: ~38h (≈13h por persona) — 1 semana intensa: NT3 + NT4

SPRINT 5 — Usuarios y Roles + Cierre Final (NT5) | S15 (18/05–24/05)	
🎯 **Objetivo:** Implementar el esquema de seguridad Oracle con 4 roles diferenciados, crear usuarios, asignar privilegios y preparar todos los entregables para la sustentación.	⏱ **Duración:** 1 semana 📦 **Entregable:** Script NT5 + Documento sustentación + Entrega final

EPIC-6: Usuarios y Roles (NT5)

ID	Tarea	Responsable	Est.	Prioridad	Descripción
NT5-1	**Crear los 4 roles Oracle con privilegios**	Daniel	5h	🔴 **Crítica**	*CREATE ROLE para: ROL_ADMIN (CRUD todas las tablas + ejecutar todos los SPs), ROL_ANALISTA (SELECT todas + reportes + vistas mat.), ROL_SOPORTE (SELECT usuarios/perfiles/pagos + INSERT/UPDATE pagos + SP_CAMBIAR_PLAN), ROL_CONTENIDO (CRUD contenido/temporadas/episodios/géneros + SELECT reproducciones).*
NT5-2	**Crear usuarios Oracle — uno por rol**	Diego	3h	🔴 **Crítica**	*CREATE USER admin_qflix, analista_qflix, soporte_qflix, contenido_qflix con contraseñas seguras. Asignar CREATE SESSION. Asignar rol correspondiente con GRANT.*
NT5-3	**GRANT de privilegios detallados por rol**	Cristhian	4h	🔴 **Alta**	*GRANT específicos sobre cada tabla para cada rol según la matriz del proyecto. Incluir privilegios sobre procedimientos almacenados y vistas materializadas. Documentar con comentarios cada GRANT.*

ID	Tarea	Responsable	Est.	Prioridad	Descripción
NT5-4	**Demostración de restricción de acceso**	Daniel	4h	🔴 **Alta**	*Conectar con cada usuario y ejecutar operación NO permitida. Ejemplo: soporte_qflix intenta DELETE en CONTENIDO → ORA-01031: insufficient privileges. Documentar captura del error para cada rol.*
NT5-5	**Crear PROFILE de recursos Oracle**	Diego	3h	🔴 **Alta**	*CREATE PROFILE perfil_quindioflix LIMIT: SESSIONS_PER_USER 3, IDLE_TIME 30, FAILED_LOGIN_ATTEMPTS 5, PASSWORD_LOCK_TIME 1/24. Asignar a los usuarios. Justificar cada límite.*
NT5-6	**Documento de sustentación**	Cristhian	8h	🔴 **Alta**	*Máximo 10 páginas. Incluir: justificación de decisiones de diseño del MER, justificación del esquema de tablespaces, análisis de índices (antes/después), escenario de concurrencia documentado con capturas, conclusiones del grupo.*
NT5-7	**Revisión general y corrección de scripts**	Diego	5h	🔴 **Alta**	*Verificar que el paquete completo funciona: borrar flyway_schema_history en Oracle, arrancar proyecto con Spring Boot — Flyway re-ejecuta V1 a V9 en orden. Corregir cualquier error. Verificar que los .sql también corren standalone en SQL Developer para la sustentación.*
NT5-8	**Empaquetar entregables finales**	Daniel	3h	🟡 **Media**	*Organizar carpeta final: E1-documento modelo negocio, E2-MER.png, E3-create_tables.sql, E4-datos_prueba.sql, E5-nucleo1.sql, E6-nucleo2.sql, E7-nucleo3.sql, E8-nucleo4.sql + capturas, E9-nucleo5.sql, E10-sustentacion.pdf.*

Mapa de Dependencias — Sprint 5

SPRINT 5 — Dependencias entre tareas (NT5 + Cierre)		
Tarea	**Tipo**	**Dependencia**
NT5-1	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — No depende de ninguna otra tarea dentro del sprint. Hacer primero.
NT5-1	🟣 **BLOQUEA**	🟣 Bloquea a NT5-2, NT5-3 y NT5-4 — Sin los roles definidos no se pueden crear usuarios, hacer GRANTs ni demostrar restricciones.
NT5-2	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE NT5-1 — Los usuarios se crean con un rol ya existente; NT5-1 debe estar listo.
NT5-2	🟣 **BLOQUEA**	🟣 Bloquea a NT5-4 y NT5-5 — Sin usuarios no se puede demostrar restricción de acceso ni asignar el PROFILE.
NT5-3	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE NT5-1 y NT5-2 — Los GRANTs se otorgan sobre roles y usuarios ya existentes.
NT5-3	🟣 **BLOQUEA**	🟣 Bloquea a NT5-4 — La demostración de acceso restringido necesita los GRANTs correctamente configurados.
NT5-4	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE NT5-1, NT5-2 y NT5-3 — La demo conecta con cada usuario y verifica que los privilegios funcionan; todo lo anterior debe estar completo.
NT5-5	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE NT5-2 — El PROFILE se asigna a los usuarios Oracle ya creados; NT5-2 debe estar listo.
NT5-6	🟢 **INDEPENDIENTE**	🟢 INDEPENDIENTE — El documento de sustentación puede redactarse en paralelo mientras los demás avanzan con los scripts.
NT5-7	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE NT5-3, NT5-4 y NT5-5 — La revisión general ejecuta el NT5 completo; roles, usuarios, GRANTs y PROFILE deben estar listos.
NT5-7	🟣 **BLOQUEA**	🟣 Bloquea a NT5-8 — Solo se empaqueta cuando la revisión general pasa sin errores.

NT5-8	🔴 **BLOQUEANTE**	🔴 BLOQUEANTE NT5-6 y NT5-7 — El empaquetado final requiere el documento de sustentación terminado y la revisión general aprobada.

Total Sprint 5: ~35h (≈12h por persona)

---

6. Entregables del Proyecto

#	Entregable	Formato	Sprint
E1	Documento modelo de negocio (actores, procesos, 10+ reglas, restricciones)	Word/PDF	Sprint 1
E2	Modelo Entidad-Relación (MER) completo y profesional	PNG/PDF	Sprint 1
E3	Script CREATE TABLE con restricciones y comentarios	.sql	Sprint 1
E4	Script inserción de datos de prueba (asimétricos)	.sql	Sprint 1-2
E5	Script Núcleo 1: consultas avanzadas + vistas mat. + fragmentación	.sql	Sprint 2
E6	Script Núcleo 2: PL/SQL — cursores, SPs, funciones, excepciones, triggers	.sql	Sprint 3
E7	Script Núcleo 3: 3 transacciones + concurrencia	.sql	Sprint 4
E8	Script Núcleo 4: índices + EXPLAIN PLAN + capturas	.sql + img	Sprint 4
E9	Script Núcleo 5: roles, usuarios Oracle, GRANT, PROFILE	.sql	Sprint 5
E10	Documento de sustentación (máx. 10 pág.)	Word/PDF	Sprint 5

---

7. Criterios de Evaluación por Núcleo

Núcleo	Peso	Criterios
NT1: Consultas avanzadas	25%	Parametrizadas, PIVOT/UNPIVOT, ROLLUP/CUBE/GROUPING SETS, Vistas materializadas, Fragmentación
NT2: PL/SQL	30%	Cursores, Procedimientos, Funciones, Excepciones, Triggers
NT3: Transacciones	15%	3 transacciones críticas + escenario de concurrencia
NT4: Índices	10%	4 índices justificados + análisis EXPLAIN PLAN
NT5: Usuarios y roles	10%	3 roles diferenciados + usuarios Oracle + GRANT + PROFILE
Calidad general	10%	Modelo de datos, datos de prueba asimétricos, scripts comentados, doc. sustentación

---

8. Notas Técnicas del Entorno

Stack tecnológico: Spring Boot + Flyway + Oracle XE local (sin frontend, sin despliegue):

Componente	Detalle
BD	Oracle XE 21c — local en cada máquina (ruta de instalación varía por equipo)
Datafiles propios	Carpeta local de cada integrante — configurada vía DEFINE ruta_base en V1__tablespaces.sql (cada quien define su propia ruta)
Backend	Spring Boot (Java) — proyecto minimalista sin controllers ni REST
Migraciones	Flyway — ejecuta los scripts .sql en orden automáticamente al arrancar
IDE	IntelliJ IDEA — incluye Database tool window para ver Oracle directamente
Control de versiones	GitHub — cada script .sql se commitea como un archivo de migración Flyway (V1__, V2__...)
Ejecución	mvn spring-boot:run → Flyway aplica scripts nuevos → Oracle se actualiza solo
Entrega final	Los .sql también funcionan standalone en SQL Developer/SQL*Plus (requisito del profe)
Sin despliegue	Todo corre local — no se requiere AWS, GCP ni Azure
Concurrencia NT3-4	Clase Java con JDBC que abre 2 conexiones simultáneas — activar con perfil --spring.profiles.active=concurrencia

Flujo de trabajo en equipo: alguien hace push de un script nuevo → los demás hacen pull → arrancan el proyecto (mvn spring-boot:run) → Flyway detecta el script nuevo y lo ejecuta automáticamente en su Oracle local. Nadie corre scripts manualmente. Los archivos .sql son entregables válidos para el profe porque existen como archivos independientes en el repo y funcionan también en SQL Developer/SQL*Plus.

---

9. Distribución de Trabajo por Persona

Ingeniero	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5	Total
Daniel Narváez	16h	16h	22h	12h	12h	78h
Diego García	19h	13h	23h	12h	11h	78h
Cristhian Osorio	21h	12h	19h	14h	12h	78h

Distribución equilibrada: exactamente 78h por persona por persona en 8 semanas de trabajo (≈9.75h/semana). * Sprint 1 abarca 2 semanas calendario pero solo 1 semana efectiva por Semana Santa.